

React Day 1 — Complete Interview Notes

Today you covered the **core React fundamentals** and connected React with your Spring Boot backend. These are the notes you should revise for interviews.

1. What is React?

React is a JavaScript library used to build user interfaces, especially single-page applications (SPAs).

React allows us to build UI using reusable **components**.

Interview answer

React is a JavaScript library for building user interfaces. It uses a component-based architecture, where the UI is divided into reusable components. React also provides features such as state management, props, event handling, and hooks like `useState` and `useEffect`.

2. React Component

A component is a reusable piece of UI.

Example:

```
function Staff() {  
  return (  
    <div>  
      <h2>Rahul</h2>  
      <p>Department: Cardiology</p>  
      <p>Role: Nurse</p>  
    </div>  
  );  
}  
  
export default Staff;
```

We can use it inside `App.jsx`:

```
import Staff from './components/Staff';

function App() {
  return (
    <div>
      <h1>Staff Management System</h1>

      <Staff />
    </div>
  );
}
```

Interview answer

A React component is a reusable and independent piece of UI. Components help us divide a large application into smaller, manageable parts.

3. JSX

JSX allows us to write HTML-like syntax inside JavaScript.

Example:

```
function App() {
  return (
    <div>
      <h1>Staff Management System</h1>
      <p>Welcome</p>
    </div>
  );
}
```

JSX is **not exactly HTML**. It is syntax that gets transformed into JavaScript.

4. JavaScript inside JSX

We use {} to execute JavaScript expressions inside JSX.

Example:

```
const name = "Rahul";
```

```
return (  
  <h2>{name}</h2>  
);
```

Output:

Rahul

Another example:

```
const count = 10;
```

```
<h2>{count}</h2>
```

5. Props

Props = properties.

Props are used to pass data from a parent component to a child component.

Parent

```
<Staff  
  name="Rahul"  
  department="Cardiology"  
  role="Nurse"  
>
```

Child

```
function Staff(props) {  
  return (  
    <div>  
      <h2>{props.name}</h2>  
      <p>Department: {props.department}</p>  
      <p>Role: {props.role}</p>  
    </div>  
  );  
}
```

You can also destructure props:

```
function Staff({ name, department, role }) {  
  return (  
    <div>  
      <h2>{name}</h2>  
      <p>Department: {department}</p>  
      <p>Role: {role}</p>  
    </div>  
  );  
}
```

Important interview point

Props are generally:

- passed from parent → child
- read-only inside the child
- used to make components reusable

Interview answer

Props are used to pass data from a parent component to a child component. Props are read-only and help make components reusable.

6. Rendering Multiple Components with `map()`

You created this:

```
const staffList = [  
  {  
    id: 1,  
    name: "Rahul",  
    department: "Cardiology",  
    role: "Nurse"  
  },  
  {  
    id: 2,  
    name: "Priya",  
    department: "Emergency",  
    role: "Doctor"  
  },  
  {  
    id: 3,
```

```
    name: "Amit",  
    department: "Neurology",  
    role: "Nurse"  
  }  
];
```

Then:

```
{staffList.map((staff) => (  
  <Staff  
    key={staff.id}  
    name={staff.name}  
    department={staff.department}  
    role={staff.role}  
  />  
))}
```

map() goes through every object in the array and creates a Staff component.

7. Why do we use `key`?

You used:

```
key={staff.id}
```

React needs a unique key when rendering a list.

Example:

```
{staffList.map((staff) => (  
  <Staff  
    key={staff.id}  
    name={staff.name}  
    department={staff.department}  
    role={staff.role}  
  />  
))}
```

Why?

The key helps React identify which list item has:

- been added

- removed
- changed

Interview answer

A key is a unique identifier used by React when rendering lists. It helps React efficiently identify which elements have changed.

Prefer:

```
key={staff.id}
```

rather than:

```
key={index}
```

when a stable ID is available.

8. useState

You learned state using:

```
const [staffCount, setStaffCount] = useState(0);
```

This gives us two things:

staffCount → current state

setStaffCount → function used to update state

Initial value:

```
0
```

Example

```
const [count, setCount] = useState(0);
```

Update:

```
setCount(count + 1);
```

React updates the state and re-renders the component.

9. Important correction about `useState`

You previously said:

`staffCount` is existing count and `setStaffCount` is a new staff count.

More accurately:

```
const [staffCount, setStaffCount] = useState(0);
```

means:

- `staffCount` → current state value
- `setStaffCount` → state updater function

It is **not a new count variable**.

10. Why not directly change state?

Don't do:

```
staffCount = staffCount + 1;
```

Instead:

```
setStaffCount(staffCount + 1);
```

React needs the setter to know that the state changed and trigger a re-render.

11. Event Handling

You created:

```
function handleClick() {  
  setStaffCount(staffCount + 1);  
  console.log("Button clicked");  
}
```

Then:

```
<button onClick={handleClick}>
  Add Staff
</button>
```

Important

You correctly learned the difference between:

```
onClick={handleClick}
```

and:

```
onClick={handleClick()}
```

Correct

```
onClick={handleClick}
```

This passes the function to React.

Usually wrong here

```
onClick={handleClick()}
```

This calls the function immediately during rendering.

Interview answer

React uses event handlers such as `onClick` to respond to user actions. We normally pass the function reference to the event handler.

12. `useEffect`

You learned:

```
useEffect(() => {
  console.log("App loaded");
}, []);
```

`useEffect` is a React Hook used to perform side effects in a component.

Examples of side effects:

- API calls
- fetching data
- subscriptions

- timers
 - interacting with external systems
-

13. Empty Dependency Array

You used:

```
useEffect(() => {  
  console.log("App loaded");  
}, []);
```

The empty array:

```
[]
```

means the effect is intended to run after the component's initial render.

This is commonly used for initial API fetching.

14. Fetching Data from Spring Boot

This was one of the most important things you learned today.

Your React frontend called:

```
http://localhost:8080/api/staff
```

Your Spring Boot backend returned JSON:

```
[  
  {  
    "id": 2,  
    "name": "Rahul",  
    "department": "Cardiology",  
    "role": "Nurse"  
  }  
]
```

React receives this data and displays it.

15. fetch()

You used:

```
fetch("http://localhost:8080/api/staff")
```

fetch() sends an HTTP request.

Then:

```
.then(response => response.json())
```

converts the response body into JavaScript data.

Then:

```
.then(data => {  
  setStaffList(data);  
})
```

stores the received data in React state.

16. Complete API Flow

This is very important for your interview.

```
React  
↓  
fetch()  
↓  
GET /api/staff  
↓  
Spring Boot Controller  
↓  
Service  
↓  
Repository  
↓  
Hibernate/JPA  
↓  
MySQL  
↓  
Database data
```

↓
Spring Boot
↓
JSON response
↓
React
↓
setStaffList()
↓
UI updates

Interview answer

When React needs staff data, it sends a GET request to the Spring Boot API using fetch. The controller receives the request and calls the service. The service uses the repository, which uses JPA/Hibernate to retrieve data from MySQL. Spring Boot returns the data as JSON, and React stores it in state using setStaffList, causing the UI to re-render.

This is an **excellent interview explanation** because it connects your frontend and backend knowledge.

17. API Data → State → UI

You created:

```
const [staffList, setStaffList] = useState([]);
```

Initially:

```
staffList = []
```

Then API returns data.

You do:

```
setStaffList(data);
```

Now:

```
staffList = [  
  {  
    id: 2,
```

```
    name: "Rahul",  
    department: "Cardiology",  
    role: "Nurse"  
  }  
]
```

React re-renders.

Then:

```
staffList.map(...)
```

creates the UI.

18. Your Final API-based React Structure

The important code you learned today is essentially:

```
import { useState, useEffect } from 'react';  
import './App.css';  
import Staff from './components/Staff';  
  
function App() {  
  
  const [staffList, setStaffList] = useState([]);  
  
  useEffect(() => {  
  
    fetch("http://localhost:8080/api/staff")  
      .then(response => response.json())  
      .then(data => {  
        setStaffList(data);  
      })  
      .catch(error => {  
        console.log("Error fetching staff:", error);  
      });  
  
  }, []);  
  
  return (  
    <div>
```

```
<h1>Staff Management System</h1>

{staffList.map((staff) => (
  <Staff
    key={staff.id}
    name={staff.name}
    department={staff.department}
    role={staff.role}
  />
))}

</div>
);
}

export default App;
```

19. Loading State

You also started learning:

```
const [loading, setLoading] = useState(true);
```

The idea is:

```
API request starts
↓
loading = true
↓
show "Loading..."
↓
API completes
↓
loading = false
↓
show staff
```

For example:

```
const [loading, setLoading] = useState(true);
```

```
useEffect(() => {  
  fetch("http://localhost:8080/api/staff")  
    .then(response => response.json())  
    .then(data => {  
      setStaffList(data);  
      setLoading(false);  
    });  
}, []);
```

Then:

```
{loading ? (  
  <p>Loading...</p>  
) : (  
  staffList.map((staff) => (  
    <Staff  
      key={staff.id}  
      name={staff.name}  
      department={staff.department}  
      role={staff.role}  
    />  
  ))  
)}
```

20. Error State

You also started adding:

```
const [error, setError] = useState("");
```

Be careful with the spelling:



```
const [error, serError] = useState("");
```



```
const [error, setError] = useState("");
```

Then:

```
.catch(error => {  
  setError("Failed to fetch staff");  
});
```

Display:

```
{error && <p>{error}</p>}
```

21. Three Important React States

For API-based applications, you'll commonly have:

```
const [staffList, setStaffList] = useState([]);  
const [loading, setLoading] = useState(true);  
const [error, setError] = useState("");
```

Think:

staffList → What data do I have?

loading → Is the API request still running?

error → Did something go wrong?

This pattern is extremely useful in real projects.

22. React Re-rendering

When state changes:

```
setStaffList(data);
```

React re-renders the component.

For example:

Before:

```
staffList = []
```

After API:

```
staffList = [Rahul]
```

```
setStaffList(data)
  ↓
React re-renders
  ↓
staffList.map()
  ↓
Rahul appears on screen
```

23. Component-Based Architecture

Your project now has:

```
App
├── Staff
```

Later your project can become:

```
App
├── Navbar
├── StaffList
│   └── Staff
├── StaffForm
└── Footer
```

This is one of the major advantages of React.

24. Important React Hooks You Learned

Today:

`useState`

Used for state.

```
const [staffList, setStaffList] = useState([]);
```

`useEffect`

Used for side effects such as API calls.


```
useEffect(() => {  
  fetch(...);  
}, []);
```

These two hooks are **very important for React interviews**.

25. React Day 1 Interview Questions

Q1. What is React?

React is a JavaScript library for building user interfaces using reusable components.

Q2. What is a component?

A component is a reusable and independent piece of UI.

Q3. What are props?

Props are used to pass data from a parent component to a child component. They are read-only.

Q4. What is state?

State is data managed by a component that can change over time. When state changes, React re-renders the component.

Q5. What is `useState`?

`useState` is a React Hook used to create and manage state in a functional component.

Example:

```
const [count, setCount] = useState(0);
```

Q6. What is `useEffect`?

`useEffect` is a React Hook used to perform side effects, such as API calls, after rendering.

Q7. Why do we use `[]` in `useEffect`?

An empty dependency array means the effect is intended to run after the initial render rather than after every render.

Q8. Why do we use `map()` in React?

We use `map()` to transform an array of data into multiple React elements.

Q9. Why is `key` required when rendering lists?

A key gives each list item a stable identity so React can efficiently determine which items changed.

Q10. What is `fetch()`?

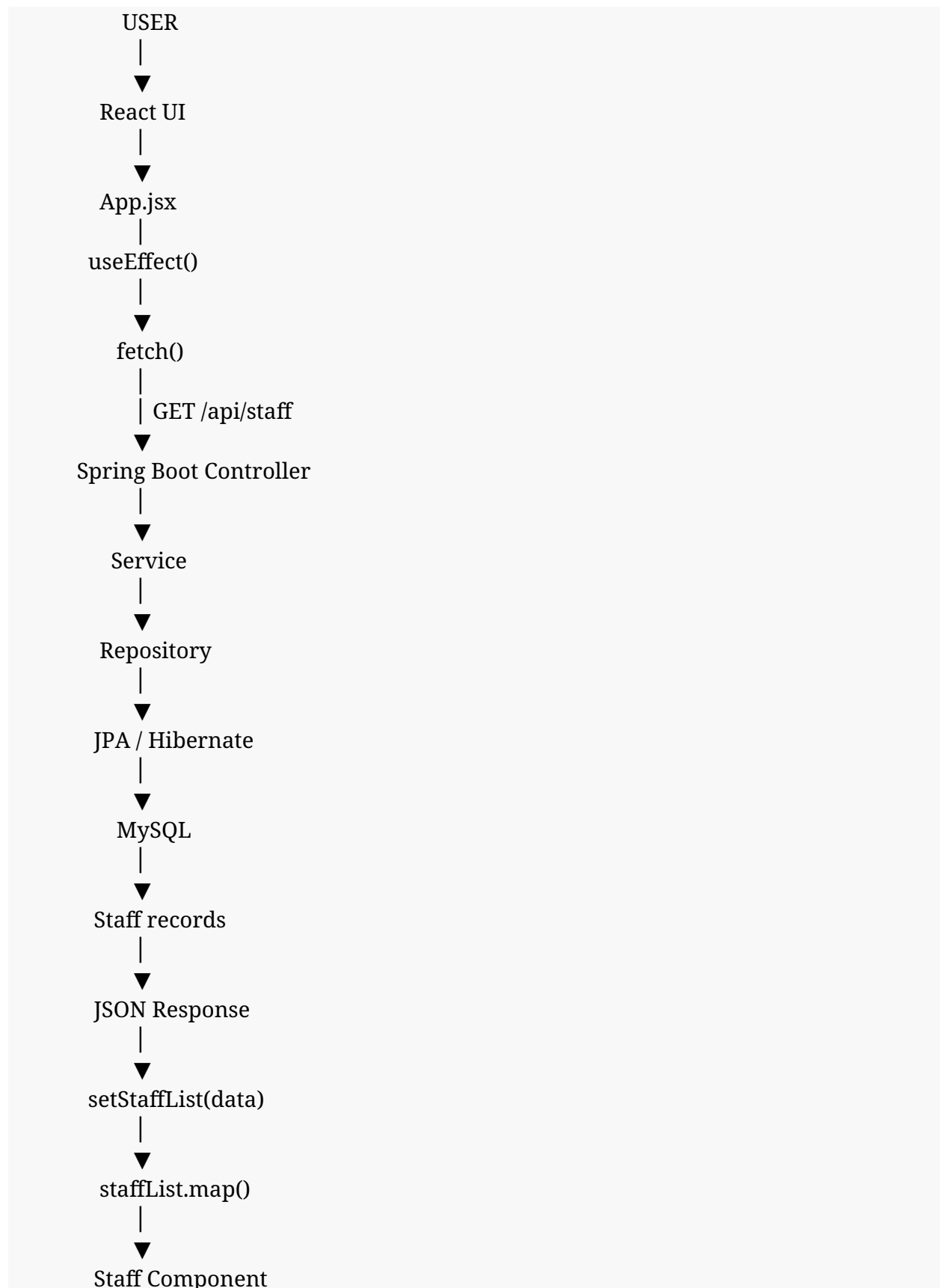
`fetch()` is a browser API used to make HTTP requests and retrieve data from an API.

Q11. How did you connect React with Spring Boot?

I used `fetch()` in React to call my Spring Boot REST API. Spring Boot retrieves data through the service and repository layers using JPA/Hibernate and MySQL, returns JSON, and React stores that response in state and renders it using `map()`.

26. Your Full Project Flow

You should be able to explain this without looking at notes:





27. Most Important Things to Remember from Day 1

React fundamentals

React

- Components
- JSX
- Props
- State
- Events
- map()
- key
- useState
- useEffect

API integration

fetch()
↓
response.json()
↓
setStaffList(data)
↓
map()
↓
UI

Your architecture

React
↓
REST API
↓
Spring Boot
↓
Service
↓

Repository
↓
JPA/Hibernate
↓
MySQL

Day 1 Interview-Level Summary

If the interviewer asks “**What did you work on in React?**”, you can say:

“I learned the fundamentals of React including functional components, JSX, props, state management using useState, event handling, rendering lists using map() with unique keys, and side effects using useEffect. I also integrated my React frontend with a Spring Boot REST API using fetch(). The API retrieves staff data from MySQL through Spring Data JPA and Hibernate, returns JSON, and I store that data in React state and dynamically render it using components.”

That’s a strong answer for a **beginner/fresher React interview**.

One correction to keep in mind

You have now learned enough to understand this distinction:

Props → data passed into a component
State → data managed by the component
useEffect → side effects/API calls
useState → state management
map() → render array data
key → identify list items
fetch() → communicate with backend API

Day 1 React is complete. Your next React session can move into the next level: **forms, controlled inputs, POST requests, and connecting a React form to your Spring Boot POST /api/staff endpoint.**